

7 Linux Route Command Examples (How to Add Route in Linux)

by LAKSHMANAN GANAPATHY on APRIL 30, 2012

In the 1st part of the IP Routing series, we learned the [fundamentals of Linux IP Routing](#).

Route command is used to show/manipulate the IP routing table. It is primarily used to setup static routes to specific host or networks via an interface.

In this article we will see how to manipulate the routing tables in Linux using route command.

We'll first explain how routing is done with some basic route command examples, and then we'll explain using a sample network architecture about how to setup routes in your network.

I. How Routing is Done?

1. Display Existing Routes

route command by default will show the details of the kernel routing table entries. In this example, the ip-address of the system where the route command is being executed is 192.168.1.157

```
$ route
Kernel IP routing table
Destination      Gateway          Genmask         Flags Metric Ref    Use I
192.168.1.0      *                255.255.255.0   U        0      0      0 e
```

The above command shows that if the destination is within the network range 192.168.1.0 – 192.168.1.255, then the gateway is *, which is 0.0.0.0.

When packets are sent within this IP range, then the MAC address of the destination is

found through ARP Protocol and the packet will be sent to the MAC address.



If you don't know what ARP is, you should first understand [how ARP protocol works](#).

In order to send packets to destination which is not within this ip range, the packets will be forwarded to a default gateway, which decides further routing for that packet. We will see this shortly.

By default route command displays the host name in its output. We can request it to display the numerical IP address using -n option as shown below.

```
$ route -n
Kernel IP routing table
Destination      Gateway          Genmask         Flags Metric Ref    Use I
192.168.1.0      0.0.0.0         255.255.255.0  U        0      0      0 e
0.0.0.0          192.168.1.10   0.0.0.0        UG       0      0      0 e
```

2. Adding a Default Gateway

We can specify that the packets that are not within the network has to be forwarded to a Gateway address.

The following route add command will set the default gateway as 192.168.1.10.

```
$ route add default gw 192.168.1.10
```

Now the route command will display the following entries.

```
$ route
Kernel IP routing table
Destination        Gateway            Genmask           Flags Metric Ref    Use I
192.168.1.0         *                  255.255.255.0     U        0      0      0 e
default             gateway.co.in      0.0.0.0           UG       0      0      0 e
```

Now we have just added a default gateway to our machine. To verify whether it is working properly, ping some external host (for example, google.com) to send ICMP packet.

```
$ ping www.google.com
```

The following is the sequences of events that happens when the above ping command is executed.

1. First it will query the DNS server to obtain the ip-address of google.com (for example: 74.125.236.34)
2. The destination address (74.125.236.34) is not within the network range.
3. So, in Layer-3 (IP header) the DESTINATION IP will be set as “74.125.236.34”.
4. In Layer-2, the DESTINATION MAC address will be filled in as the MAC address of the default gateway (192.168.1.10's MAC). The MAC will be found by using ARP as described earlier.
5. When the packet is sent out, the network switch (which works on Layer-2), send the packet to the default gateway since the destination MAC is that of the gateway.
6. Once the gateway receives the packet, based on its routing table, it will forward the packets further.

The above 2 examples would have given a good idea about how routing is done within a network. Now we will see other command line options available with route command.

3. List Kernel's Routing Cache Information

Kernel maintains the routing cache information to route the packets faster. We can list the

kernel's routing cache information by using the -C flag.

```
$ route -Cn
Kernel IP routing cache

Source          Destination      Gateway          Flags Metric Ref    Use I
192.168.1.157    192.168.1.51     192.168.1.51     0      0      1 e
192.168.1.157    74.125.236.69    192.168.1.10     0      0      0 e
.
.
.
```

4. Reject Routing to a Particular Host or Network

Sometimes we may want to reject routing the packets to a particular host/network. To do that, add the following entry.

```
$ route add -host 192.168.1.51 reject
```

As you see below, we cannot access that particular host (i.e .51 host that we just rejected).

```
$ ping 192.168.1.51
connect: Network is unreachable
```

However we can still access other hosts in the network (for example, .52 host is still accessible).

```
$ ping 192.168.1.53
PING 192.168.1.53 (192.168.1.53) 56(84) bytes of data.
64 bytes from 192.168.1.53: icmp_seq=1 ttl=64 time=7.77 ms
```

If you want to reject an entire network (192.168.1.1 – 192.168.1.255), then add the following entry.

```
$ route add -net 192.168.1.0 netmask 255.255.255.0 reject
```

Now, you cannot access any of the host in that network (for example: .51, .52, .53, etc.)

```
$ ping 192.168.1.51
connect: Network is unreachable

$ ping 192.168.1.52
connect: Network is unreachable

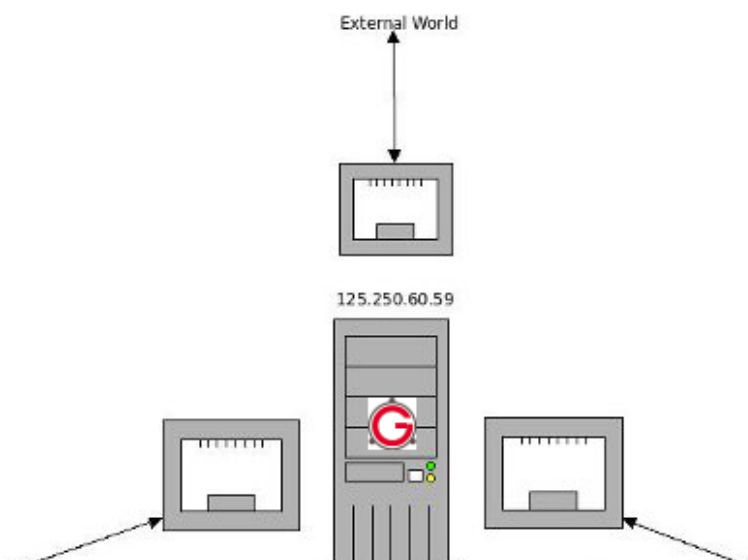
$ ping 192.168.1.53
connect: Network is unreachable
```

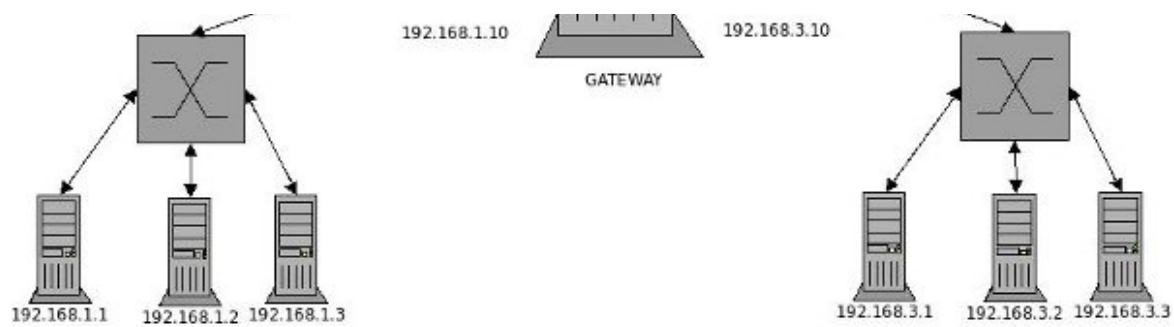
II. A Sample Network Architecture (to understand routing)

Let us use the following sample network architecture for the rest of the examples.

In the diagram below, we have 2 individual networks (192.168.1.0 and 192.168.3.0, with subnet mask of 255.255.255.0).

We also have a “GATEWAY” machine with 3 network cards. 1st card is connected to 192.168.1.0, 2nd card is connected to 192.168.3.0, and the 3rd card is connected to the external world.





5. Make 192.168.3.* Accessible from 192.168.1.*

Now we need to add a routing entry such that we are able to ping 192.168.3.* series ip-addresses from 192.168.1.* series. The common point we have is the GATEWAY machine.

So, on each machine in 192.168.1.* network a default gateway will be added as shown below.

```
$ route add default gw 192.168.1.10
```

Now when 192.168.1.1 pings 192.168.3.1, it will go to the GATEWAY via 192.168.1.10.

In GATEWAY, add the following routing entry.

```
$ route add -net 192.168.3.0 netmask 255.255.255.0 gw 192.168.3.10
```

Now all the packets addressed to 192.168.3.* network will be forwarded via the 192.168.3.10 interface, which then delivers the packets to the addressed machine.

6. Make 192.168.1.* Accessible from 192.168.3.*

It is very similar to what we did earlier.

So, on each machine in 192.168.3.* network a default gateway will be added as shown below.

```
$ route add default gw 192.168.3.10
```

In GATEWAY, add the following routing entry.

```
$ route add -net 192.168.1.0 netmask 255.255.255.0 gw 192.168.1.10
```

Now 192.168.3.* machines can ping 192.168.1.* machines.

7. Allow Internet Access (External World)

In the previous two example, we have interconnected the 2 different networks.

Now we need to access the internet from these 2 different networks. For that, we can add a default routing (when no routing rule matches) to the 125.250.60.59 which is connected to the external world as follows.

```
$ route add default gw 125.250.60.59
```

This is how it works:

1. Now when you try to access the internet (for example: ping google.com) from any of these machines (for example, from 192.168.3.2), the following is the sequence of events that happens.
2. Since the destination (google.com) is not within 3.* series, it will be forwarded to GATEWAY via 3.10 interface
3. In GATEWAY, it checks whether the destination is within 1.* range. In this example, it is not.
4. It then checks whether the destination is within 2.* range. IN this example, it is not
5. Finally, it takes the default route to forward the packets (i.e using the 125.250.60.59 interface, which is connected to the external world).